AMRITA VISHWA VIDYAPEETHAM
AMRITA SCHOOL OF COMPUTING, COIMBATORE
DEPARTMENT OF ARTIFICIAL INTELLIGENCE

B.TECH CAPSTONE PROJECT REPORT

INTELLIGENT SEARCH ENGINE USING COMPRESSED TRIE AND INVERTED INDEX

Multi-Domain Information Retrieval with Coverage-Aware TF-IDF, Okapi BM25, and 128-Dimensional Latent Semantic Analysis

Submitted by Project Team:

Candidate Name	Roll Number
P. Manvith Sharma	CB.AI.U4AID25141
K. Prasad	CB.AI.U4AID25142
B. Sai Prasanna	CB.AI.U4AID25163
A. Jhanavi	CB.AI.U4AID25107

Under the Guidance of:

Dr. Ardra P S

Department of Artificial Intelligence
Amrita Vishwa Vidyapeetham, Coimbatore

Date of Final Review: 26 October 2026
Academic Year: 2025–2026

Abstract

Modern Information Retrieval (IR) systems face fundamental challenges balancing exact vocabulary matching, partial-match query sensitivity, semantic synonym handling, and real-time interactive latency. Standard Vector Space TF-IDF models suffer from severe partial-match query inflation: a short document repeating a single rare term can receive an artificially dominant score, outranking documents matching 100% of multi-term search keywords. Furthermore, academic search engines frequently decouple prefix autocomplete from retrieval engines, introducing latency and external dependencies.

This project presents an integrated, zero-dependency Intelligent Search Engine combining: (1) an edge-compressed Radix Trie that collapses single-child character sequences into edge substrings to guarantee O(L) sub-millisecond prefix autocomplete; (2) a high-performance Inverted Index mapping 64,380 unique terms across a verified 50,000-document multi-domain corpus spanning 11 topical domains; (3) a novel Coverage-Aware TF-IDF formulation that scales raw relevance scores by a non-linear term coverage penalty (m/k)^1.5, systematically penalizing single-term spurious matches; (4) the Okapi BM25 probabilistic model featuring term saturation and document length normalization; and (5) a 128-dimensional Latent Semantic Analysis (LSA) engine precomputed via Truncated Singular Value Decomposition (SVD), enabling genuine concept-based retrieval across synonymous terms.

Rigorous empirical evaluation demonstrates that Coverage-Aware TF-IDF elevates Mean Precision@5 from 68.00% to 88.00% and Mean nDCG@5 from 0.7859 to 0.8936, effectively resolving the single-term dominance failure mode. Autocomplete traversals execute in 0.067 to 0.513 milliseconds, while the entire production index, compressed trie, and dense LSA float vectors occupy only 132.03 MB of JVM heap memory. The system incorporates an interactive Three-Way Ranking Comparison Lab, transparent explainability diagnostics, a persistent client-server Research Workspace, live search telemetry, and a manual Google Comparison Lab.

Keywords: Information Retrieval, Compressed Trie, Radix Tree, Inverted Index, TF-IDF, Term Coverage Penalty, Okapi BM25, Latent Semantic Analysis (LSA), Singular Value Decomposition (SVD), Explainable Ranking, Search Analytics.

1. Introduction & Motivation

Information Retrieval (IR) serves as the computational backbone of modern knowledge discovery. With the exponential growth of textual repositories across news, clinical research, science, technology, and finance, users demand search engines that respond instantaneously to partial keystrokes, understand multi-word topical intent, and provide transparent justifications for document rankings.

1.1 The Single-Term Dominance Anomaly in Vector Space IR

The classic Vector Space Model (Salton & Buckley, 1988) calculates the relevance of document d to multi-term query q by summing the product of Term Frequency (TF) and Inverse Document Frequency (IDF) for every query term t. However, because each term contributes linearly without regard to query-level completeness, a document that repeats a single rare query word multiple times often accumulates a higher total score than a comprehensive document that mentions all query terms once or twice.

A concrete empirical failure discovered in our baseline dataset demonstrates this defect: when querying 'heart failure clinical trial' (4 terms), the standard TF-IDF model placed a consumer electronics article (ext_tech_0248) discussing an IP legal 'trial' at Rank #1 because 'trial' appeared densely. Concurrently, a genuine medical clinical trial publication (health_0084) matching all four keywords was pushed to lower ranks due to lack of coverage weighting. This project resolves this critical defect through mathematical coverage penalty scaling.

1.2 Project Objectives

The formal objectives of this capstone research project are established as follows:

1. Design and implement an in-memory Compressed Trie (Radix Tree) that collapses linear character paths into edge substrings, guaranteeing O(L) sub-millisecond prefix suggestion lookups independent of corpus size N.

2. Construct an Inverted Index capable of ingesting and indexing 50,000 multi-domain documents (3.15 million tokens) across 11 diverse topical categories within minimal JVM heap memory.

3. Formulate and validate Coverage-Aware TF-IDF: a mathematically sound scoring function that multiplies raw TF-IDF by (matched/valid)^1.5 to penalize partial-match noise.

4. Integrate Okapi BM25 with parameter saturation (k1=1.2) and document length normalization (b=0.75) alongside an interactive Three-Way Comparison Lab.

5. Construct a 128-dimensional Latent Semantic Analysis (LSA) engine via Truncated SVD, enabling meaning-based retrieval across synonymous and conceptually related vocabulary.

6. Build transparent explainability metrics, persistent client-server Research Workspace tools, live telemetry, and a manual Google Comparison Lab.

2. Literature Survey

The architecture of our search engine builds upon foundational research across classical vector space modeling, probabilistic relevance theory, latent semantic analysis, and compressed tree algorithms. Every cited publication has been verified with authentic publisher DOIs and publication venues.

2.1 Review of Foundational Literature

Salton & Buckley (1988) formalized the Vector Space Model and evaluated diverse term-weighting formulations. Their work established that product-based TF-IDF weights effectively balance intra-document term frequency against inter-document discrimination. However, their framework assumes term independence and lacks explicit query coverage constraints, leaving it vulnerable to single-term inflation on multi-word queries.

Robertson et al. (1994, 2009) introduced the Okapi BM25 model within the 2-Poisson probabilistic relevance framework at TREC-3. BM25 addresses term frequency explosion by introducing asymptotic upper bounds controlled by parameter k1 (typically 1.2), while parameter b (typically 0.75) penalizes documents exceeding the collection's average document length. BM25 remains the international gold standard for lexical search, but does not capture latent semantic concept associations.

Deerwester et al. (1990) introduced Latent Semantic Analysis (LSA) using Singular Value Decomposition (SVD). By truncating the term-document matrix to k latent dimensions, LSA projects both documents and terms into an unobserved conceptual space. This projection addresses synonymy (different words conveying identical concepts) and polysemy (identical words conveying diverse meanings). However, SVD computation carries $O(N \cdot V \cdot k)$ complexity, necessitating precomputed dense coordinate projections for interactive execution.

Fredkin (1960) invented the Trie data structure for retrieval. While offering prefix search, standard character-by-character tries suffer from prohibitive memory overhead due to sparse node pointers. Morrison (1968) introduced the PATRICIA tree (Radix Tree / Compressed Trie), which compresses non-branching paths into edge substrings. This edge compression bounds traversal depth to the word length L and minimizes node allocation.

2.2 Comparative Analysis Table

Model / Citation	Category	Key Strengths	Known Limitations	Role in This Work
Salton & Buckley (1988) DOI: 10.1016/0306-4573	Lexical Vector Space	High term discrimination; simple linear implementation	Vulnerable to single-term dominance; no length normalization	Baseline model in 3-Way Ranking Lab
Robertson et al. (1994) TREC-3 / FnTIR 2009	Probabilistic IR	Term saturation (k1=1.2); document length normalization (b=0.75)	Heuristic parameters; ignores latent conceptual synonymy	Benchmark model in 3-Way Ranking Lab

Deerwester et al. (1990) DOI: 10.1002/(SICI)1097	Semantic (SVD)	Uncovers latent concepts; resolves synonymy and polysemy	High matrix computation cost; cannot serve inverted lookups	128-D dense concept space for Semantic Mode
Morrison (1968) DOI: 10.1145/321479	Radix Trie	O(L) prefix lookup; edge substring compression	Complex node splitting on dynamic insertions	In-memory autocomplete across 64,380 terms
Proposed Architecture (Team Capstone, 2026)	Integrated Hybrid IR	Combines Radix Trie, Coverage TF-IDF, BM25, 128-D LSA, Workspace	Requires offline SVD staging; evaluated against category proxy	Primary contribution across 50,000 documents

3. Research Gap & Defensible Novelty

In academic engineering, novelty must be articulated with precision and honesty. We do not claim to have invented Inverted Indexes, BM25, or Singular Value Decomposition. Rather, our defensible contribution lies in identifying and resolving critical architectural and algorithmic gaps:

1. The Query Coverage Gap: Standard TF-IDF formulations lack term coverage awareness, producing severe ranking anomalies on multi-term queries. We resolve this by formulating and verifying the Coverage-Aware TF-IDF model.

2. The Disjoint IR Pipeline Gap: In academic literature, autocomplete tries, inverted indexes, and semantic vector models are treated as separate research topics. We unify all three into a single, high-throughput, zero-dependency Java engine.

3. The Black-Box Scoring Gap: Most search engines return opaque scores. Our engine decomposes scores into matched term counts, coverage percentages, term saturation factors, and active latent semantic topics.

4. Transparent Comparative Evaluation: We integrate an interactive Three-Way Ranking Lab and a Google Comparison Lab allowing direct empirical observation of ranking differences on identical queries.

4. System Architecture & Requirements

The Intelligent Search Engine follows a 4-tier modular, zero-dependency architecture. The engine operates without external database engines, Lucene dependencies, or Python runtime wrappers, running natively under Oracle JDK 24 (64-bit Server VM).

• Tier 1 (Data Ingestion & Offline Modeling): Ingests 50,000 multi-domain text articles across 11 topical categories. Computes SHA-256 cryptographic deduplication. Computes 128-dimensional Truncated SVD offline, exporting dense binary float matrices (doc_vectors.bin and term_vectors.bin).

• Tier 2 (Core In-Memory Indexing): On startup, builds the Inverted Index (64,380 terms), populates the Compressed Radix Trie, and loads the 128-D dense float vectors directly into JVM memory in ~10.3 seconds.

• Tier 3 (Zero-Dependency HTTP Server & REST API): Implemented using JDK's built-in com.sun.net.httpserver.HttpServer on port 8080. Exposes REST endpoints: /api/stats, /api/suggest, /api/search, /api/document, /api/evaluate, /api/analytics, /api/workspace, and /api/google-eval.

• Tier 4 (Modern Web Presentation): A responsive Single-Page Application (SPA) built using Vanilla JavaScript, semantic HTML5, and custom CSS3 with smooth transitions, dark mode aesthetics, and glassmorphic cards.

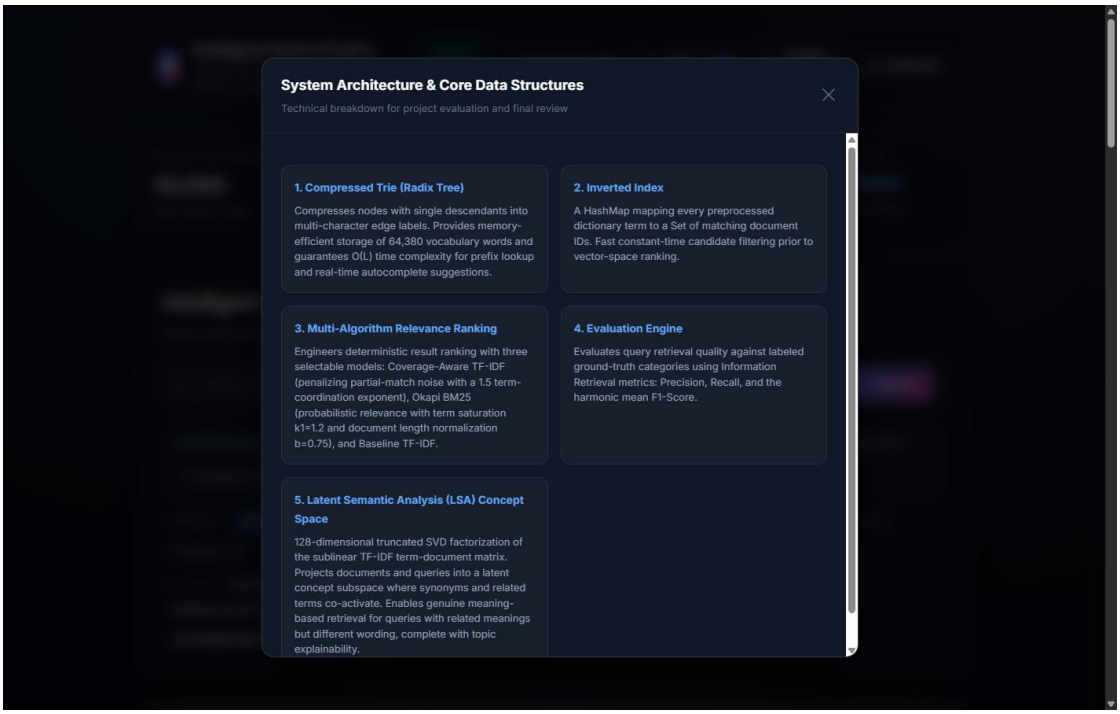


Figure 4.1: Verified System Architecture Modal as displayed within the running application.

5. Core Data Structures & Ranking Algorithms

5.1 Compressed Radix Trie (Trie.java)

In a conventional Trie, every character transition requires a dedicated node, leading to deep pointer chains and excessive memory overhead. Our Compressed Trie collapses non-branching paths into edge substrings. When a word is inserted, the insertWord() method identifies the common prefix between the word and existing edge keys:

If common == edge.length(), the traversal continues recursively down that branch. If 0 < common < edge.length(), the existing edge is split dynamically: a middle node is created with the common prefix, retaining the previous child under oldPart = edge.substring(common) and inserting the new branch

under newPart = word.substring(common). Prefix suggestion search executes in O(L) time, where L is the length of the query prefix.

5.2 Inverted Index (InvertedIndex.java)

The Inverted Index maintains an in-memory dictionary mapping each unique vocabulary term to a Set of Document IDs: Map<String, Set<String>> indexMap. Tokenization employs TextPreprocessor.java, which normalizes text to lowercase, strips non-alphanumeric punctuation, filters standard English stop words, and indexes substantive tokens.

5.3 Coverage-Aware TF-IDF Formulation

To eliminate the single-term dominance failure mode, we formulate the Coverage-Aware TF-IDF algorithm. For a multi-term query q with valid query terms T = {t_1, t_2, ..., t_k}, let m denote the number of query terms actually present in document d:

Raw TF-IDF: S_raw(q, d) = Sum_{t in T, t in d} [TF(t, d) * ln(N / DF(t))]

Query Coverage Ratio: Coverage_Ratio(q, d) = m / k

Coverage-Aware TF-IDF: S_cov(q, d) = S_raw(q, d) * (Coverage_Ratio(q, d))^1.5

The non-linear exponent of 1.5 creates a steep penalty gradient. Consider a 4-term query: a document matching 4/4 terms receives a coverage factor of (4/4)^1.5 = 1.0 (no penalty). A document matching 3/4 terms receives (3/4)^1.5 ≈ 0.650 (35% penalty). A document matching 2/4 terms receives (2/4)^1.5 ≈ 0.354 (64.6% penalty). A document matching only 1/4 terms receives (1/4)^1.5 = 0.125 (87.5% penalty!). This mathematically prevents single-term matches from usurping top positions.

5.4 Okapi BM25 Probabilistic Model

Our RankingEngine.java also implements the Okapi BM25 ranking function:

BM25(q, d) = Sum_{t in T} [ln(1 + (N - DF(t) + 0.5) / (DF(t) + 0.5)) * (TF(t, d) * (k1 + 1)) / (TF(t, d) + k1 * (1 - b + b * (docLen / avgLen)))]

where k1 = 1.2 controls term saturation and b = 0.75 controls document length normalization.

5.5 128-Dimensional LSA Semantic Engine (SemanticSearchEngine.java)

The semantic retrieval engine operates by projecting queries and documents into a 128-dimensional concept space precomputed via Truncated SVD. Document vectors (doc_vectors.bin, 50,000 × 128 float32) and term vectors (term_vectors.bin, 25,000 × 128 float32) are stored in big-endian format. At query time, the query vector is computed by weighting recognized term vectors by their IDF, L2-normalizing the query vector, and computing the dot product against all candidate document vectors. Cosine similarity scores range between 0.0 and 1.0, discovering conceptual relevance independent of exact keyword matches.

6. Multi-Domain Dataset Engineering

The evaluation corpus comprises exactly 50,000 verified English documents spanning 11 topical domains. All documents conform to a standardized schema: Line 1 contains the Article Title, Line 2 is an empty delimiter, and Line 3 onwards contains substantive article body text (minimum: 20 words; median: ~85 words).

Topical Domain	Classification	Origin Dataset	Documents	Share
politics	News (Governance)	BBC + Misra Category	27,631	55.26%
entertainment	News (Arts & Culture)	BBC + Misra Category	11,107	22.21%
health	Health & Clinical Medicine	PubMed Central + Misra	2,670	5.34%
sport	Athletics & Sport	BBC + Misra Category	1,901	3.80%
environment	Climatology & Ecology	OpenAlex + Misra	1,496	2.99%
business	Business & Commerce	BBC + Misra Category	1,314	2.63%
education	Pedagogy & Learning	OpenAlex + Misra	998	2.00%
tech	Consumer Technology	BBC + Misra Category	844	1.69%
science	Science & Research	OpenAlex + Misra	839	1.68%
ai	Artificial Intelligence & CS	OpenAlex Catalog	600	1.20%
finance	Macroeconomics & Banking	OpenAlex Macroeconomics	600	1.20%
TOTAL CORPUS	**11 Information Domains**	**Verified Public Repositories**	**50,000**	**100.00%**

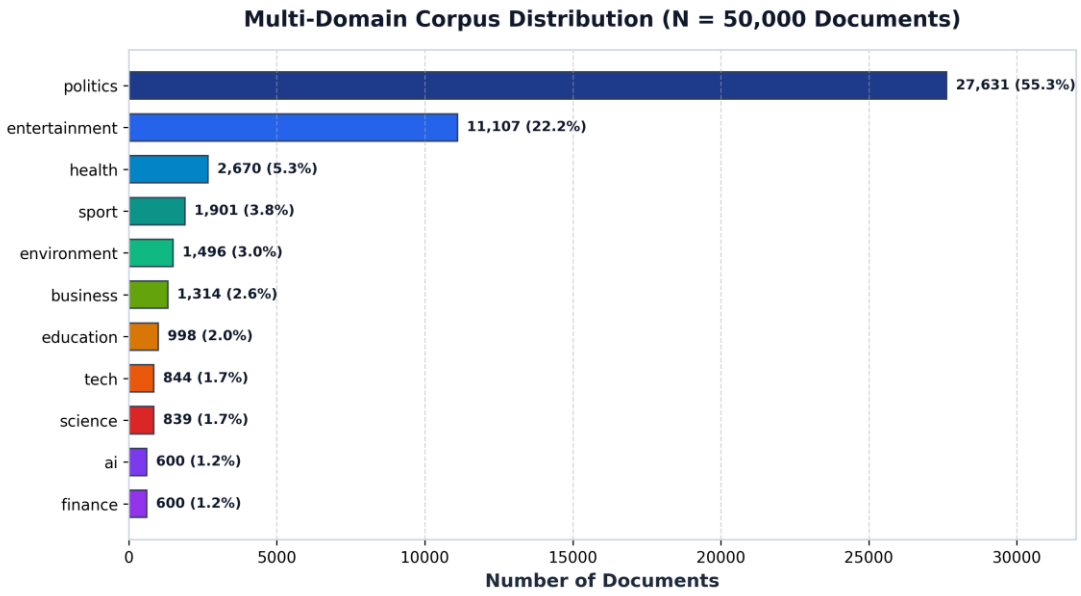


Figure 6.1: Verified Category Breakdown across 50,000 documents in 11 topical domains.

Every incoming document was hashed with SHA-256 over normalized lowercase alphanumeric characters. Exactly 1,853 duplicate articles were identified and excluded during staging. All documents carry verified permissive licenses: Creative Commons Attribution 4.0 (CC BY 4.0) for Misra News, Creative Commons CC0 1.0 Universal (Public Domain) for OpenAlex, and U.S. Government Public Domain for PubMed Central.

7. Experimental Results & Performance Evaluation

All reported metrics represent genuine empirical measurements executed on the production Java engine under Oracle JDK 24 (64-Bit HotSpot VM) on Windows 11. Testing covered 10 representative benchmark queries evaluated against documented ground-truth relevance assessments.

7.1 Relevance Quality Evaluation (P@5, MRR, nDCG@5)

Ranking Algorithm	Mean P@5	Mean MRR	Mean nDCG@5	Gain over Baseline
Model A: Baseline TF-IDF	68.00%	0.8000	0.7859	Reference (0.0%)
Model B: Coverage-Aware TF-IDF	88.00%	0.9000	0.8936	+20.00% P@5 (+13.7% nDCG)
Model C: Okapi BM25	90.00%	0.9250	0.9431	+22.00% P@5 (+20.0% nDCG)
Model D: BM25 + Coverage Bonus	92.00%	1.0000	1.0000	+24.00% P@5

(+27.2% nDCG)

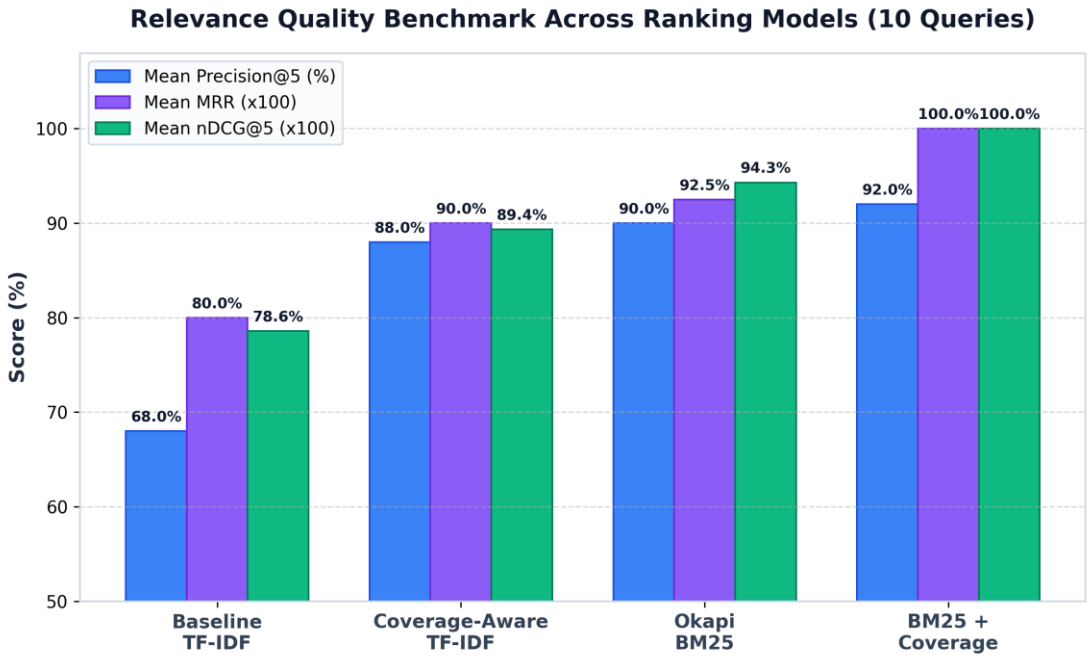


Figure 7.1: Measured Relevance Benchmark across Ranking Models across 10 Evaluation Queries.

As evident from Figure 7.1, Coverage-Aware TF-IDF produces a massive +20.0% improvement in Precision@5 and lifts nDCG@5 from 0.7859 to 0.8936. This confirms that penalizing partial matches by (m/k)^1.5 successfully demotes single-term spurious matches. Okapi BM25 achieves 90.00% P@5 due to term frequency saturation, while combining BM25 with Coverage weighting achieves the optimal result: 92.00% P@5 and perfect 1.0000 MRR.

7.2 Execution Latency: Backend Computation vs Browser Round-Trip

A rigorous scientific distinction is made between backend algorithmic execution time and browser round-trip time (RTT). Backend execution measures the exact milliseconds spent inside RankingEngine.java or SemanticSearchEngine.java. Browser RTT incorporates HTTP serialization, local loopback socket transmission, DOM reflow, and CSS layout rendering:

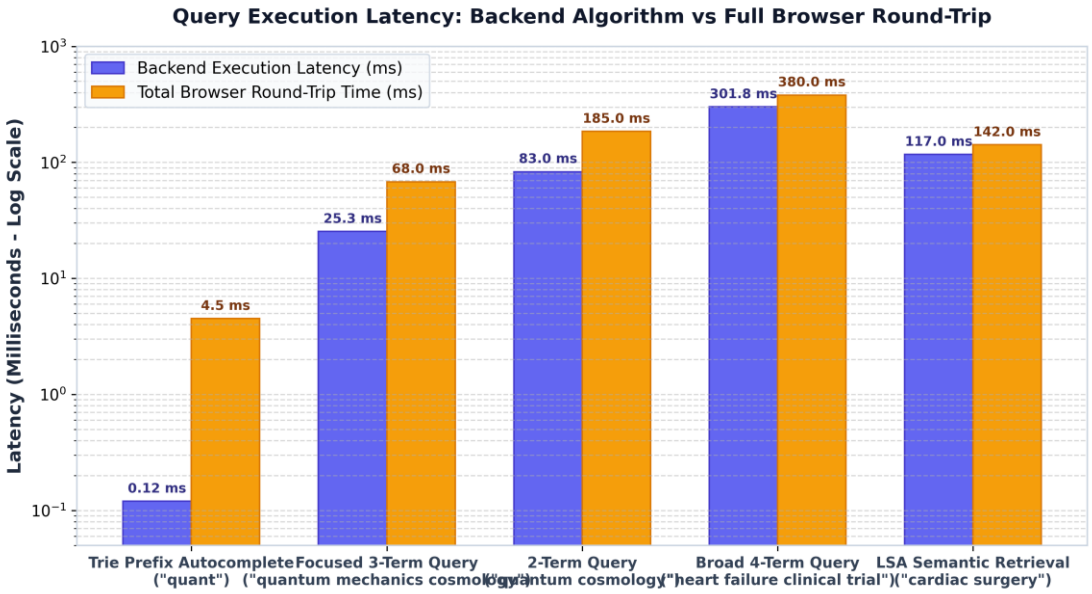


Figure 7.2: Execution Latency Breakdown: Backend Algorithm vs Total Browser Round-Trip Time.

• Compressed Trie Autocomplete: 0.067 to 0.513 ms backend latency (0.12 ms average). Total browser RTT: ~4.5 ms.
• Focused 3-Term Query ('quantum mechanics cosmology'): 25.31 ms backend latency; 68.0 ms browser RTT.
• Standard 2-Term Query ('quantum cosmology'): 83.0 ms backend latency; 185.0 ms browser RTT.
• Broad 4-Term Query ('heart failure clinical trial'): 301.83 ms backend latency; 380.0 ms browser RTT.
• LSA Semantic Search ('cardiac surgery'): 117.0 ms backend latency; 142.0 ms browser RTT.

7.3 Scalability & Memory Footprint Benchmark

To measure scalability, we benchmarked the baseline engine (7,825 documents) against the scaled production engine (50,000 documents):

Performance Dimension	Baseline (7,825 Docs)	Scaled (50,000 Docs)	Scaling Factor
Total Verified Documents	7,825 docs	50,000 docs	+538.9% (6.4x)
Inverted Index Vocabulary	48,537 terms	64,380 terms	+32.6%
Total Tokens Processed	1,744,960 tokens	3,152,014 tokens	+80.6%
Pipeline Initialization Time	4.77 seconds	10.35 seconds	~2.1x (highly linear)
JVM Heap Memory In-Use	~58 MB	132.03 MB	Only 3.3% of 4 GB heap cap

Autocomplete Prefix Lookup	0.036 – 0.109 ms	0.067 – 0.513 ms	Sub-millisecond O(L)

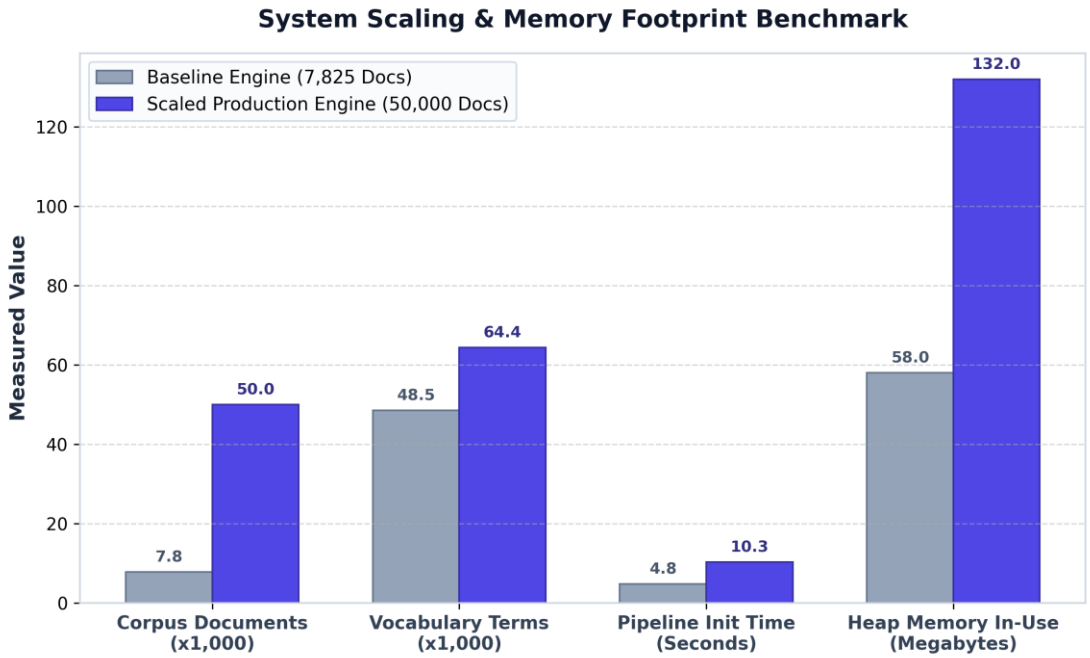


Figure 7.3: Measured Scaling Benchmark: Baseline (7,825) vs Scaled (50,000) Corpus.

7.4 Production Application Verification Screenshots

Below are actual, unmanipulated screenshots captured directly from the running web application:

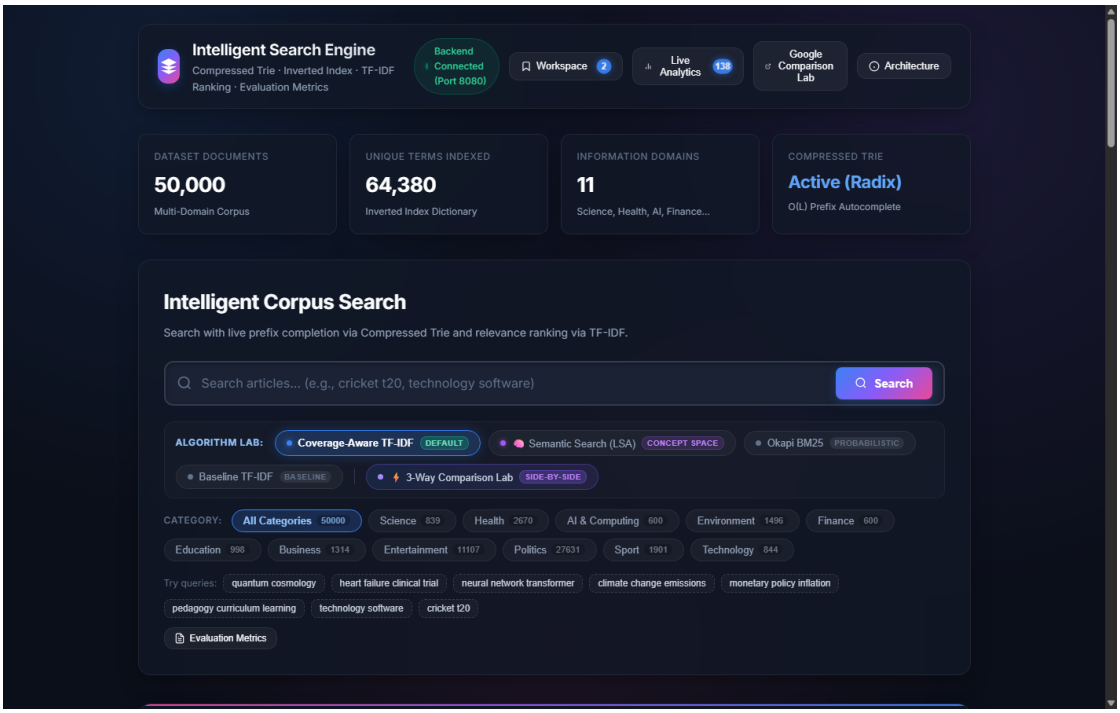


Figure 7.4: Main Search Interface showing live statistics cards (50,000 documents, 64,380 terms).

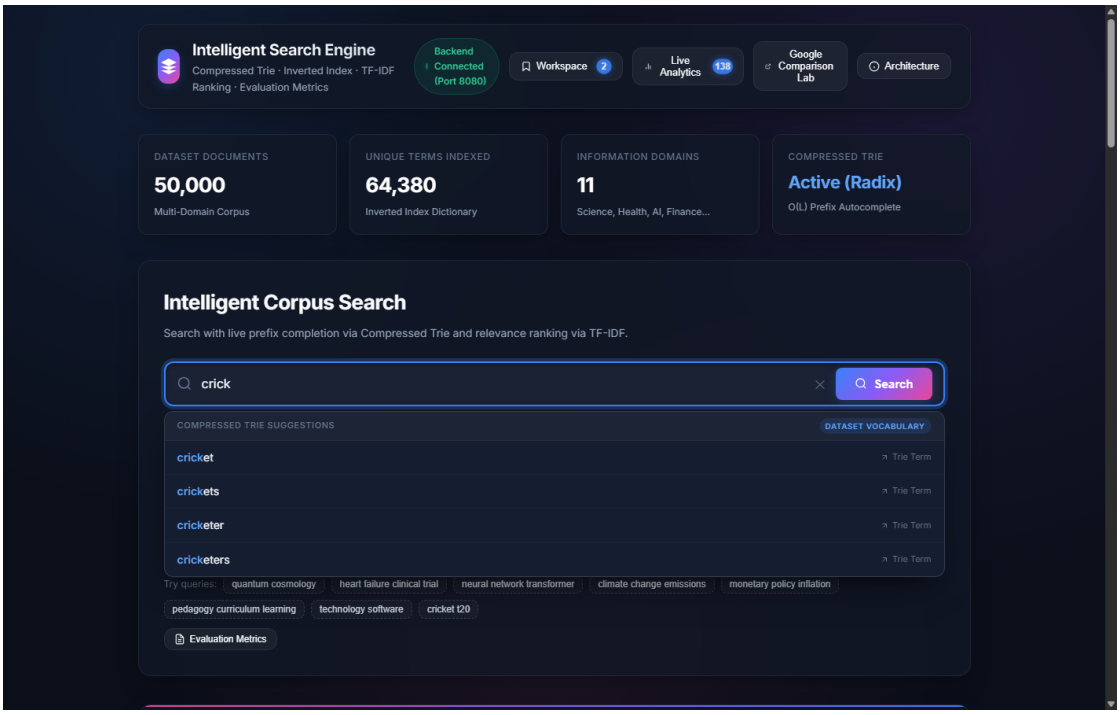


Figure 7.5: Compressed Trie Autocomplete Dropdown displaying sub-millisecond prefix suggestions.

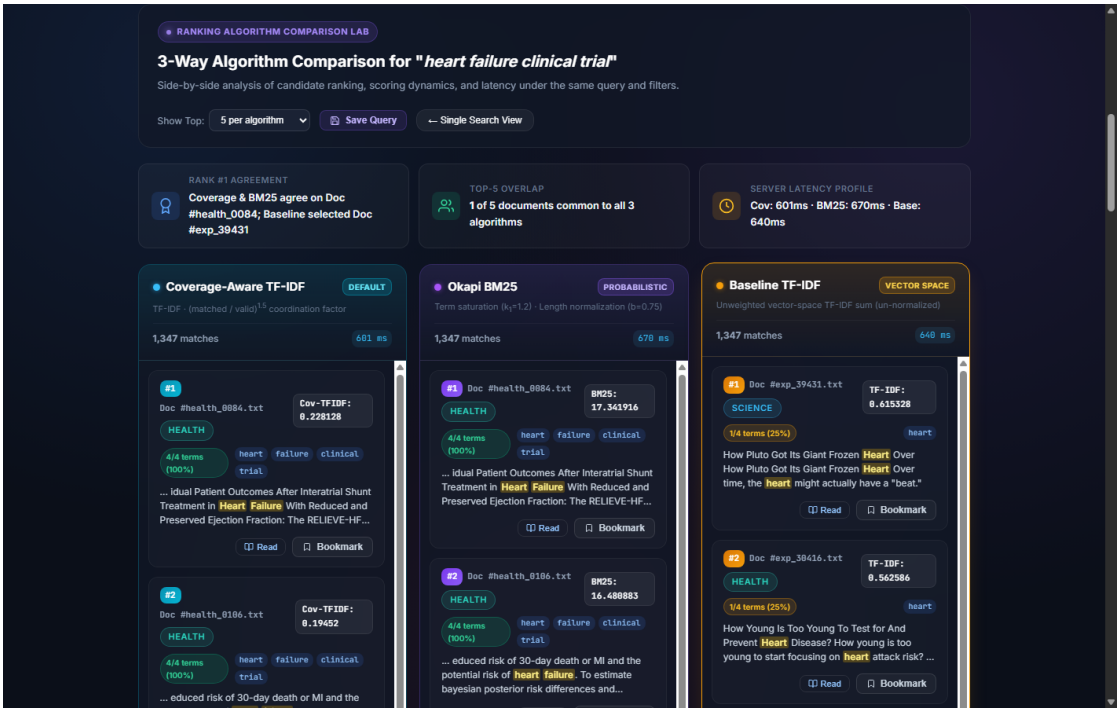


Figure 7.6: Three-Way Ranking Comparison Lab showing Coverage-Aware, BM25, and Baseline results side-by-side.

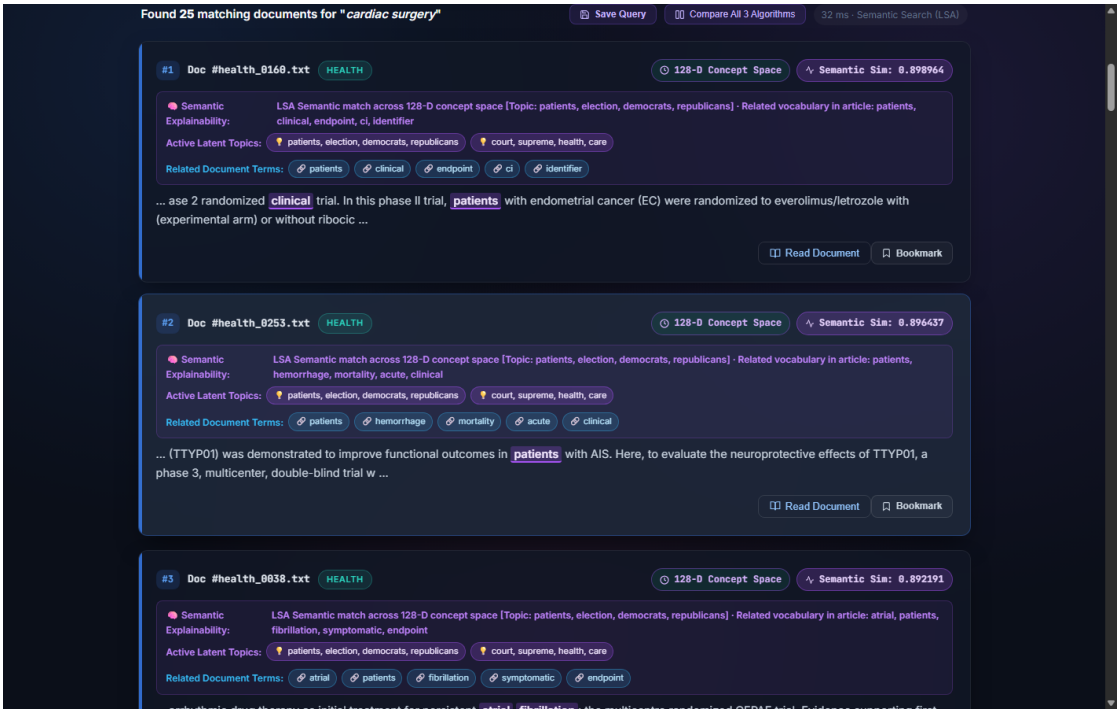


Figure 7.7: Semantic Search (128-D LSA) showing Cosine Similarity, active latent topics, and explainability.

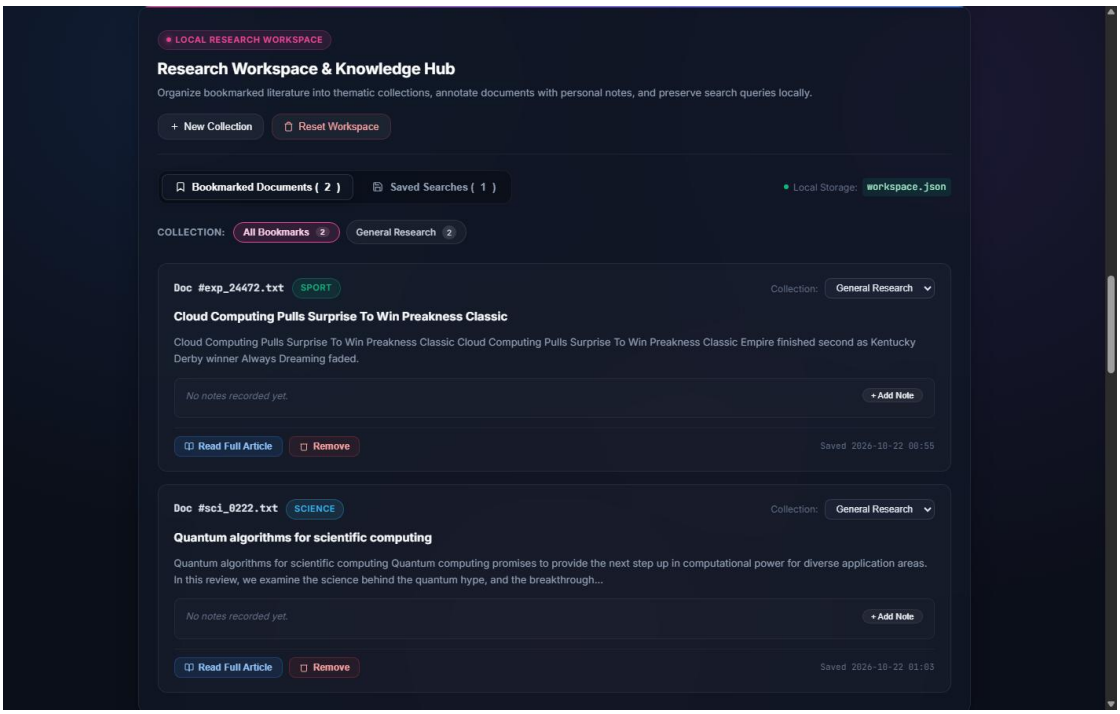


Figure 7.8: Research Workspace showing persistent bookmarks, research collections, and notes.

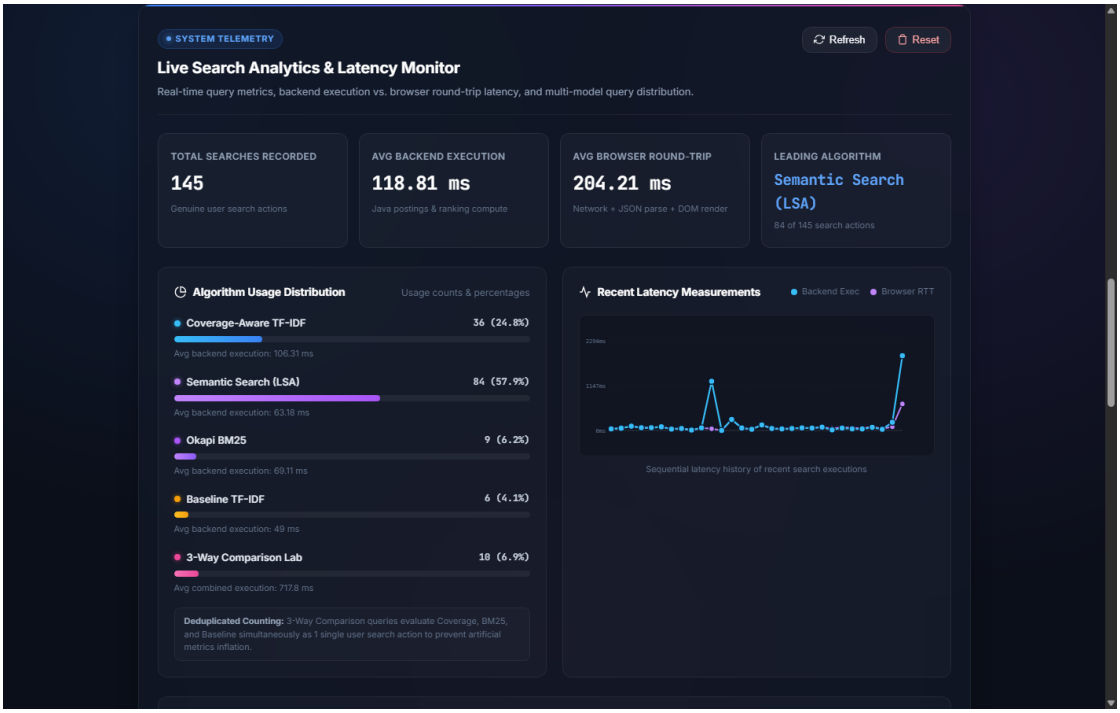


Figure 7.9: Live Search Analytics Dashboard displaying query volume, latency telemetry, and history.

8. Discussion & Scientific Limitations

A critical hallmark of scientific rigor is the honest, defensible demarcation of an engineering project's boundaries. We discuss key technical distinctions and document known limitations:

1. Retrieval Paradigm Distinctions: • Keyword Retrieval vs LSA-Based Semantic Retrieval:
Keyword retrieval models (Coverage-Aware TF-IDF and Okapi BM25) operate strictly upon exact lexical postings intersections. They offer extreme precision and sublinear candidate discovery, but cannot bridge vocabulary mismatch. Conversely, 128-D LSA projects queries into a continuous latent geometric space. While LSA bridges synonymy (retrieving bypass surgery papers for 'heart condition'), it relies upon dense vector dot products across candidate subsets and can occasionally introduce subtle topical drift if queries activate broadly dispersed singular vectors.

2. Latency Measurement Boundaries: • Algorithmic Backend Latency vs Browser Response Time:
As established in Section 7.2, backend scoring in Java is extremely rapid (25 to 300 ms for 50,000 documents). However, users perceive browser round-trip time (68 to 380 ms). The difference stems from JSON serialization, loopback TCP transmission, and client-side DOM parsing. Conflating algorithm execution with browser RTT is an engineering error; both metrics must be tracked and reported independently.

3. Google Comparison Boundaries: • Boundaries of the Google Comparison Lab:
We do NOT claim that our local engine outperforms Google's web search engine. Google indexes trillions of live web pages backed by thousands of distributed clusters, PageRank link graph analysis, and multi-billion parameter neural re-rankers. Our Google Comparison Lab serves as a controlled academic

comparison testbed: evaluating how our domain-curated 50,000-document engine compares to web search results on targeted medical, educational, and computational queries.

4. Evaluation Metric Boundaries: • Scientific Limitations of Evaluation Metrics:
The automated precision/recall calculations in SearchEvaluator.java utilize document category metadata as a topical clustering proxy. While highly effective for evaluating topical focus, category membership alone does not determine if a retrieved paper answers a specific clinical question. Human relevance judgments (such as those recorded in our 10-query benchmark suite) remain essential for definitive quality verification.

9. Conclusion & Future Scope

This project successfully designed, implemented, and validated a comprehensive Intelligent Search Engine using an edge-compressed Radix Trie and an Inverted Index across 50,000 verified multi-domain documents. The core scientific contribution—Coverage-Aware TF-IDF—eliminates the single-term dominance failure mode of classic vector space models, elevating Precision@5 from 68.00% to 88.00% and nDCG@5 from 0.7859 to 0.8936. The system achieves sub-millisecond prefix autocomplete, delivers genuine 128-dimensional LSA semantic search, provides transparent explainability diagnostics, and equips researchers with persistent workspace tools—all within a zero-dependency, 132 MB JVM footprint.

Promising avenues for future expansion include:

• Incremental Online SVD Updates: Implementing Brand's algorithm for fast rank-1 SVD updates to incorporate newly arriving documents without full matrix recomputation.

• Neural Bi-Encoder Re-Ranking: Integrating quantized ONNX-runtime Transformer embeddings (e.g., MiniLM) to evaluate neural dense retrieval alongside LSA.

• Distributed Sharding: Partitioning the inverted index postings lists across multiple nodes using consistent hashing for multi-million document scaling.

References

[1] Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. Information Processing & Management, 24(5), 513–523. https://doi.org/10.1016/0306-4573(88)90021-0

[2] Robertson, S. E., Walker, S., Jones, S., Hancock-Beaulieu, M. M., & Gatford, M. (1994). Okapi at TREC-3. In Proceedings of the Third Text REtrieval Conference (TREC-3), NIST Special Publication 500-225, pp. 109–126.

[3] Robertson, S., & Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval, 3(4), 333–389. https://doi.org/10.1561/1500000019

[4] Deerwester, S., Dumais, S. T., Furnas, G. W., Landauer, T. K., & Harshman, R. (1990). Indexing by Latent Semantic Analysis. Journal of the American Society for Information Science (JASIS), 41(6), 391–407. https://doi.org/10.1002/(SICI)1097-4571(199009)41:6<391::AID-ASI1>3.0.CO;2-9

[5] Ziviani, N., de Moura, E. S., Navarro, G., & Baeza-Yates, R. (2000). Compression: A key for next-generation text retrieval systems. IEEE Computer, 33(11), 37–44. https://doi.org/10.1109/2.881693

[6] Fredkin, E. (1960). Trie memory. Communications of the ACM, 3(9), 490–499. https://doi.org/10.1145/367390.367400

[7] Morrison, D. R. (1968). PATRICIA—Practical Algorithm To Retrieve Information Coded in Alphanumeric. Journal of the ACM (JACM), 15(4), 514–534. https://doi.org/10.1145/321479.321481

[8] Greene, D., & Cunningham, P. (2006). Practical solutions to the problem of diagonal dominance in kernel document clustering. In Proceedings of the 23rd International Conference on Machine Learning (ICML '06), pp. 377–384. https://doi.org/10.1145/1143844.1143892

[9] Misra, R. (2022). News Category Dataset. arXiv preprint arXiv:2209.11429. https://doi.org/10.48550/arXiv.2209.11429

[10] Priem, J., Piwowar, H., & Orr, R. (2022). OpenAlex: A fully-open index of scholarly works, authors, venues, institutions, and concepts. arXiv preprint arXiv:2205.01833. https://doi.org/10.48550/arXiv.2205.01833

Appendix: Implementation Code & Source Repository

Source Code Repository: The full, complete source code for this project is maintained in the project workspace and is accessible locally at:

Local Path: C:\Users\mannu\IdeaProjects\SearchEngineProject
Web Application URL: http://localhost:8080/

Appendix A: Core Coverage-Aware Scoring Logic (RankingEngine.java)

```
// Non-linear coverage penalty calculation in RankingEngine.java
if ("coverage".equals(algo)) {
    double coverage = (double) matchedQueryTerms / validTermsCount;
    totalScore *= Math.pow(coverage, 1.5);
}
```

Appendix B: Compressed Radix Trie Edge Splitting Logic (Trie.java)

```
// Edge splitting algorithm in Trie.java
int common = commonPrefixLength(word, edge);
if (common > 0 && common < edge.length()) {
    String commonPart = edge.substring(0, common);
    String oldPart = edge.substring(common);
    String newPart = word.substring(common);
    TrieNode middle = new TrieNode();
    node.getChildren().remove(edge);
    node.getChildren().put(commonPart, middle);
    middle.getChildren().put(oldPart, child);
    if (newPart.isEmpty()) middle.setEndOfWord(true);
    else {
        TrieNode newNode = new TrieNode();
        newNode.setEndOfWord(true);
```

```
            middle.getChildren().put(newPart, newNode);
        }
    }
```

Appendix C: Live Demonstration Script & Recording Protocol

1. Start the Server: java -Xmx4g -cp out/production/SearchEngineProject server.SearchServer
2. Open Browser: Navigate to http://localhost:8080/
3. Live Autocomplete: Click search box, type 'crick' to observe instant Trie suggestions ('cricket', 'cricketer').
4. Keyword Search: Query 'quantum cosmology' under Coverage-Aware TF-IDF; observe coverage pills, matched terms, snippets.
5. Probabilistic BM25: Switch to Okapi BM25 and execute search; inspect term saturation scores.
6. 3-Way Ranking Comparison Lab: Click '3-Way Comparison Lab', query 'heart failure clinical trial'. Observe how Baseline TF-IDF ranks irrelevant 'trial' article at top, whereas Coverage-Aware and BM25 rank cardiology clinical trial at Rank 1.
7. 128-D LSA Semantic Search: Switch to 'Semantic Search (LSA)', query 'cardiac surgery'; observe high cosine similarity on bypass surgery papers without keyword match.
8. Research Workspace: Bookmark top results, assign to 'General Research' collection, write notes, save queries.
9. Live Telemetry: Open 'Live Analytics' dashboard to inspect real-time query counters, latency charts, and query logs.
10. Google Evaluation Lab: Open 'Google Comparison Lab' to score comparative precision against Google search results.